

BERN02 lab: Unsupervised learning

Dimension reduction (SVD / PCA) and clustering, with a spectroscopy example

This lab accompanies the lecture on unsupervised learning (ISL chapter 12.1–12.2 and 12.4). You will

1. explore a small, well known dataset (iris) and learn why **scaling** changes the answer,
2. compute a **singular value decomposition (SVD)** by hand and understand what the pieces mean,
3. see PCA as a **rotation of the coordinate system** that minimises the unexplained variance,
4. apply SVD to a **simulated transient-absorption experiment** ($GS \rightarrow A \rightarrow B \rightarrow C \rightarrow GS$) and discover what SVD can and cannot tell you about the chemistry,
5. cluster with **k-means** and **hierarchical clustering**, and learn to pick the number of clusters with the **elbow** and **silhouette** criteria,
6. discover on the **smiley point clouds** why the choice of algorithm matters (k-means vs DBSCAN vs spectral clustering),
7. (bonus) compare linear and non-linear embeddings (PCA vs t-SNE) on handwritten digits.

Cells marked **Task** expect you to write code. In the student version the code cells contain `# your code here`. A reference solution is in `unsupervised_learning_solved.ipynb` – try first, peek later.

Convention used throughout: a data matrix `X` has one **observation per row** and one **feature (variable) per column**, exactly like a pandas DataFrame. Keep this in mind whenever you look at the shapes of `U`, `S` and `Vt`.

```
In [127... # --- Setup: run this cell first -----
# Everything below is in the standard scientific Python stack (Anaconda)
import sys, os, pathlib, warnings
warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", message="Graph is not fully connected")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import scipy.stats
import scipy.cluster.hierarchy as sch      # dendrograms and linkage
from scipy.optimize import least_squares

import sklearn.datasets
import sklearn.cluster
import sklearn.decomposition
```

```

import sklearn.preprocessing
from sklearn.preprocessing import StandardScaler
import sklearn.metrics
import sklearn.manifold
from sklearn.cluster import KMeans

import skimage.io, skimage.color, skimage.filters # image handling (rep

%matplotlib inline
plt.rcParams["figure.dpi"] = 100
sns.set_context("notebook")

RNG_SEED = 12345 # one seed for the whole notebook -> rep
np.random.seed(RNG_SEED)

# Where are the smiley images? In the course repo they live in ./Smileys
SMILEY_DIR = next((p for p in [pathlib.Path("Smileys"), pathlib.Path("sou
print("python", sys.version.split()[0], "| numpy", np.__version__, "| pan
print("Smiley images found in:", SMILEY_DIR)

```

python 3.13.13 | numpy 2.5.3 | pandas 3.0.6 | sklearn 1.9.1

Smiley images found in: Smileys

Part 1 – Getting to know the data

We use the **iris** dataset that ships with scikit-learn: 150 flowers, 4 measured lengths in cm, 3 species. The species labels exist, but we will **not** use them for learning – only for checking how well the unsupervised methods rediscover them. That is the usual situation in the lab: you measure many samples and hope the structure you find corresponds to something real.

```

In [2]: iris = sklearn.datasets.load_iris()
df_iris = pd.DataFrame(iris.data, columns=iris.feature_names)
df_iris["species"] = pd.Series(iris.target_names[iris.target], name="spec
df_iris.head()

```

```

Out[2]:

```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Task 1.1 – explore

- Read the description: `print(iris['DESCR'])`.
- Look at `df_iris.describe()`: what are the means and standard deviations of the four columns? Which column varies most?

- Use `sns.pairplot(...)` on the DataFrame (add the species as `hue`) to inspect the relations between the columns. Which pairs of variables are strongly correlated? Which single variable separates the species best by eye?

```
In [3]: # your code here
print(iris['DESCR'])
df_iris.describe()
sns.pairplot(df_iris, hue="species", diag_kind="kde")
plt.show()
```

```
.. _iris_dataset:
```

```
Iris plants dataset
```

```
**Data Set Characteristics:**
```

```
:Number of Instances: 150 (50 in each of three classes)
:Number of Attributes: 4 numeric, predictive attributes and the class
:Attribute Information:
  - sepal length in cm
  - sepal width in cm
  - petal length in cm
  - petal width in cm
  - class:
    - Iris-Setosa
    - Iris-Versicolour
    - Iris-Virginica
```

```
:Summary Statistics:
```

	Min	Max	Mean	SD	Class Correlation
sepal length:	4.3	7.9	5.84	0.83	0.7826
sepal width:	2.0	4.4	3.05	0.43	-0.4194
petal length:	1.0	6.9	3.76	1.76	0.9490 (high!)
petal width:	0.1	2.5	1.20	0.76	0.9565 (high!)

```
:Missing Attribute Values: None
:Class Distribution: 33.3% for each of 3 classes.
:Creator: R.A. Fisher
:Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
:Date: July, 1988
```

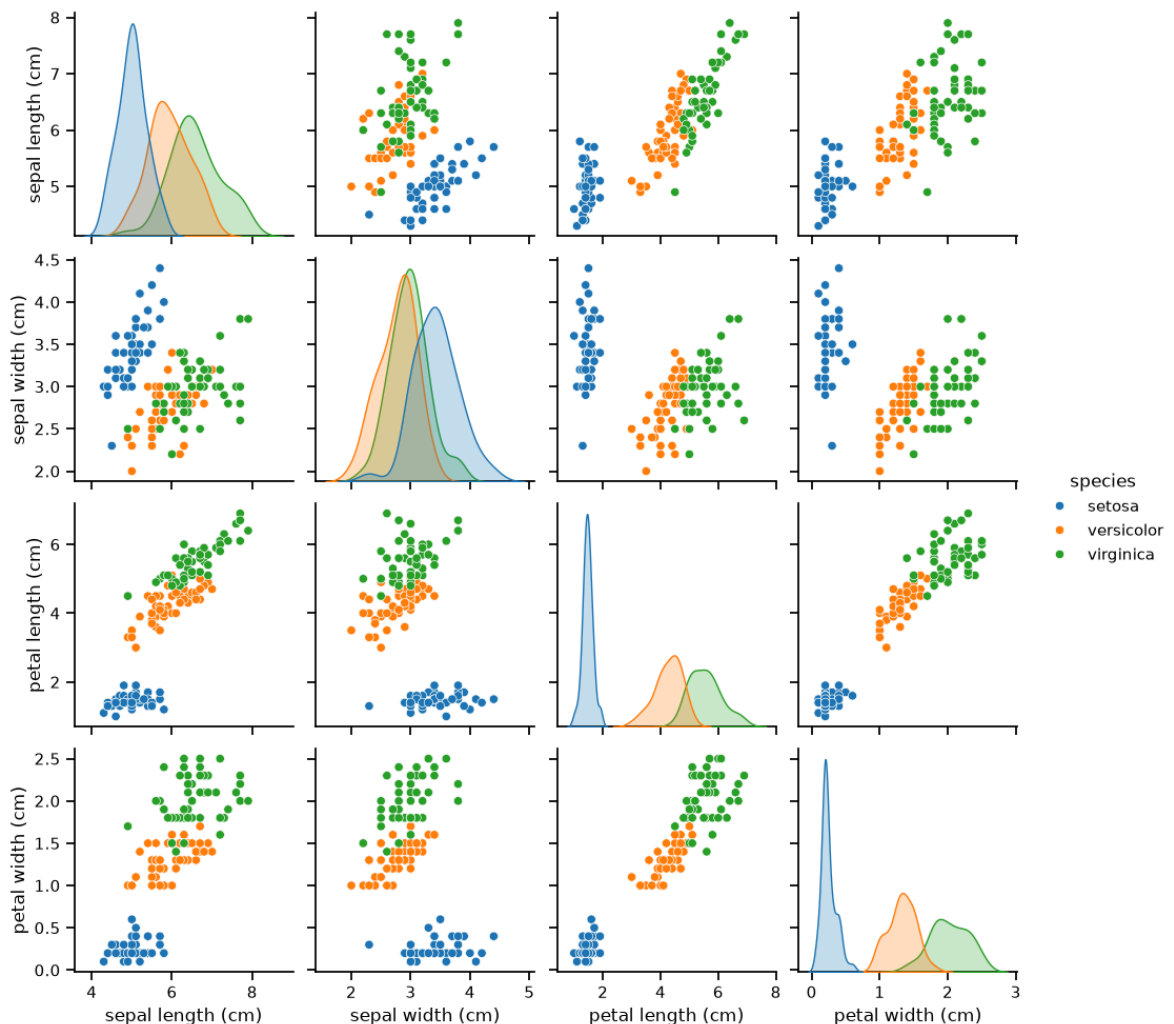
The famous Iris database, first used by Sir R.A. Fisher. The dataset is taken from Fisher's paper. Note that it's the same as in R, but not as in the UCI Machine Learning Repository, which has two wrong data points.

This is perhaps the best known database to be found in the pattern recognition literature. Fisher's paper is a classic in the field and is referenced frequently to this day. (See Duda & Hart, for example.) The data set contains 3 classes of 50 instances each, where each class refers to a type of iris plant. One class is linearly separable from the other 2; the latter are NOT linearly separable from each other.

```
.. dropdown:: References
```

- Fisher, R.A. "The use of multiple measurements in taxonomic problems" Annual Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to Mathematical Statistics" (John Wiley, NY, 1950).
- Duda, R.O., & Hart, P.E. (1973) Pattern Classification and Scene Analysis. (Q327.D83) John Wiley & Sons. ISBN 0-471-22361-1. See page 218.

- Dasarthy, B.V. (1980) "Nosing Around the Neighborhood: A New System Structure and Classification Rule for Recognition in Partially Exposed Environments". IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. PAMI-2, No. 1, 67–71.
- Gates, G.W. (1972) "The Reduced Nearest Neighbor Rule". IEEE Transactions on Information Theory, May 1972, 431–433.
- See also: 1988 MLC Proceedings, 54–64. Cheeseman et al's AUTOCLASS II conceptual clustering system finds 3 classes in the data.
- Many, many more ...



What you should have seen. Petal length and petal width are almost perfectly correlated ($r \approx 0.96$): the data are essentially lower dimensional than four. The petal variables separate the species; the sepal width does not. Note also that petal length has a standard deviation of 1.8 cm while petal width has 0.8 cm and sepal width 0.4 cm. Any method based on distances or variances will weigh the variables by these numbers – **unless we scale first**.

Part 2 – Singular value decomposition and PCA

Singular value decomposition factorises any real matrix $X \in \mathbb{R}^{m \times n}$ into

$$X = U \Sigma V^T$$

- $U \in \mathbb{R}^{m \times m}$ – the **left singular vectors**; the columns are orthonormal eigenvectors of XX^T . With our convention (rows = observations) these are patterns *over the observations*.
- Σ – diagonal, with the non-negative **singular values** $s_1 \geq s_2 \geq \dots$ on the diagonal.
- $V \in \mathbb{R}^{n \times n}$ – the **right singular vectors**; the columns are orthonormal eigenvectors of $X^T X$. These are patterns *over the features*, i.e. the directions in feature space.

`np.linalg.svd(X, full_matrices=False)` returns **U** ($m \times k$), **S** (k), and **Vt** $= V^T$ ($k \times n$) with $k = \min(m, n)$.

Connection to PCA. If the columns of X are centred (mean zero), the covariance matrix is $C = \frac{1}{m-1} X^T X = \frac{1}{m-1} V \Sigma^2 V^T$. So the right singular vectors *are* the principal component loadings, and the eigenvalues of C – the variances along the principal components – are $\lambda_j = s_j^2 / (m - 1)$. The **scores** (the coordinates of every observation in the new system) are $Z = XV = U\Sigma$.

The **proportion of variance explained** (ISL eq. 12.8 and 12.9) by component j is therefore

$$\text{PVE}_j = \frac{s_j^2}{\sum_k s_k^2}.$$

Task 2.1 – SVD on raw, centred and standardised data

Compute the SVD of the iris matrix three times:

1. on the raw data `df_iris.values`,
2. on the **centred** data (subtract the column means),
3. on the **standardised** data (subtract mean, divide by standard deviation; `sklearn.preprocessing.StandardScaler` does this).

For each case print the singular values and the PVE of every component. Then plot the cumulative PVE against the number of components for the three cases in one figure. Label the axes using the ISL terminology. What is $\sum_k s_k^2$ in each case, and what does that number mean? (Hint: for standardised data it is a nice number related to the shape of the matrix – check whether your scaler divides by m or $m - 1$.)

```
In [4]: # your code here
# X = raw data, excluding the 'species' column
X_raw = df_iris.drop(columns=['species']).values
# X = centred data (X - X_mean)
X_centered = X_raw - X_raw.mean(axis=0)
# X = standardised data ()
X_standard = StandardScaler().fit_transform(X_raw)
# Define components as the number of features in the dataset
```

```

components = np.arange(1, X_raw.shape[1] + 1)
# Set up the figure
fig, ax = plt.subplots(figsize=(8,5))
# Make a dictionary of each data set
data = {"Raw": X_raw, "Centered": X_centered, "Standardized": X_standard }

# Compute the svd for each data set using np.linalg.svd()
for type, X in data.items():
    U, s, Vt = np.linalg.svd(X, full_matrices=False)
    # Calculate proportion of variance explained (PVE)
    s_squared = s**2 # remember that the singular values are the square r
    print(f"Sum of s^2 for {type}: {s_squared.sum()}")
    pve = s_squared / s_squared.sum() # proportion of total variance expl
    cum_pve = np.cumsum(pve) # cumulative proportion of variance explaine
    ax.plot(components, cum_pve, marker='o', label=f'{type} Data')

# Plot the cumulative PVE against the number of coponents
ax.set_title('Cumulative Proportion of Variance Explained (PVE)', fontwei
ax.set_xlabel('Principal Component', fontsize=11)
ax.set_ylabel('Cumulative Proportion of Variance Explained', fontsize=11)
ax.set_xticks(components)
# Create tick marks from 0.60 to 1.00 at intervals of 0.05
ax.set_yticks(np.arange(0.60, 1.05, 0.05))
ax.set_ylim(0.7, 1.025)
ax.legend(loc="lower right")

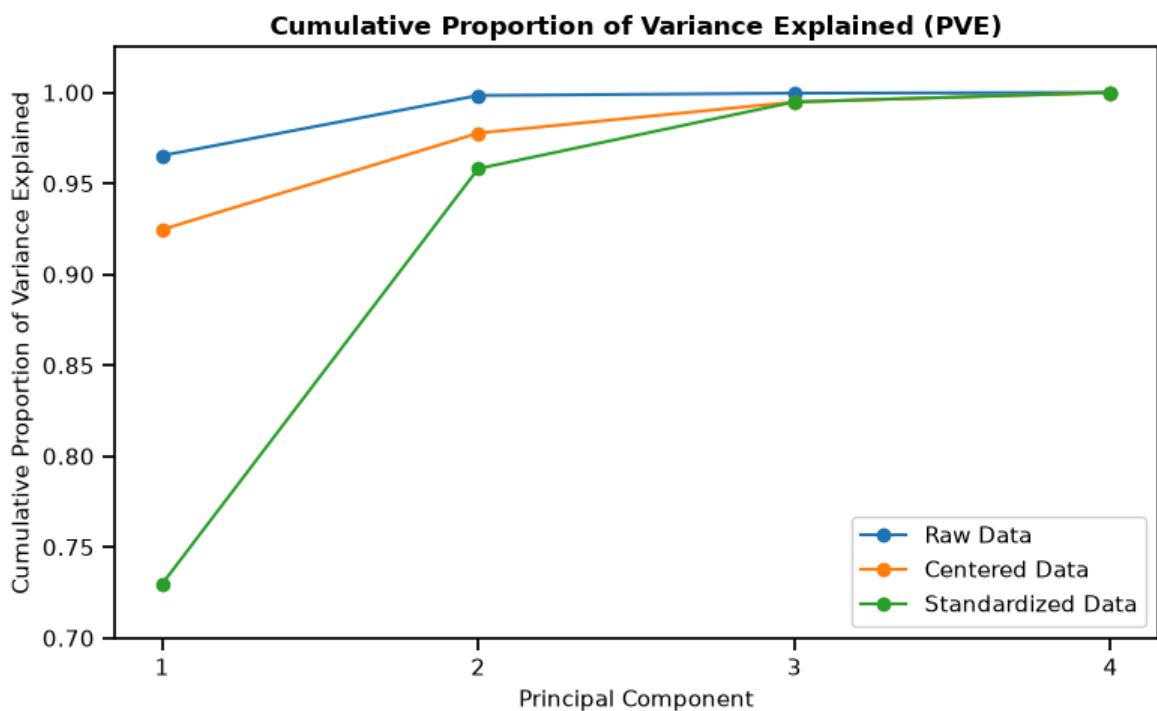
plt.tight_layout()
plt.show()

```

Sum of s^2 for Raw: 9539.289999999997

Sum of s^2 for Centered: 681.3706000000002

Sum of s^2 for Standardized: 600.0



Discussion.

- On the **raw** data the first singular value is huge and the first "component" explains 97 % – but that component is essentially the mean flower. The "variance" it explains is the offset from zero, not a variation between flowers.

This is why PCA always centres the data (and why `np.linalg.svd(df_iris)` alone is *not* PCA).

- For **centred** data $\sum s_k^2 = (m - 1) \sum_j \text{Var}(x_j) = 149 \times 4.57 \approx 681$: the total variance. The first two components explain 98 %, but petal length dominates because it has the largest variance in cm.
- For **standardised** data every column has variance 1, so $\sum s_k^2 = m \times p = 150 \times 4 = 600$ (scikit-learn's `StandardScaler` divides by the population standard deviation, `ddof=0` ; with pandas' `std()` , `ddof=1` , you get $(m - 1)p = 596$ – be aware which one you use). Now the first two components explain "only" 96 % – but each variable had the same right to contribute.

Whether to scale is a **scientific** decision, not a technical one: if all columns are the same physical quantity (e.g. absorbance at different wavelengths) you usually should *not* scale, because the variance carries information. If the columns are apples and oranges (cm, kg, pH) you must scale, otherwise the unit choice decides the result.

Task 2.2 – scores and loadings

Use the SVD of the **standardised** data. The scores $Z = U\Sigma$ are the coordinates of each flower in the principal-component system.

- Make a scatter plot of the first two scores, coloured by species (the labels are only for colouring).
- Next to it plot the same for the original first two columns of the data, and for the scores 3 and 4. Which representation separates the species best?
- Print `Vt[0]` and `Vt[1]` (the loadings) with the feature names. Which variables make up the first component? Interpret the sign pattern of the second component.

```
In [14]: # your code here
print(df_iris.columns)
# save the 4 features: 'sepal length (cm)', 'sepal width (cm)', 'petal le
feature_name = df_iris.columns[:-1]
species = df_iris["species"]

# SVD on standardized data
U, s, Vt = np.linalg.svd(X_standard, full_matrices=False)

# Scores Z = U * Sigma = X_std @ Vt.T
Z = X_standard @ Vt.T

# Plotting the original data and the PCA scores
fig, axes = plt.subplots(1,3, figsize=(18,5))
# Plot the original data
sns.scatterplot(data=df_iris, x=feature_name[0], y=feature_name[1], hue='
axes[0].set_title("Original Features (Sepal Length vs Width)")

# Plot the PCA scores 1 and 2
sns.scatterplot(x=Z[:, 0], y=Z[:, 1], hue=df_iris['species'], ax=axes[1])
axes[1].set_title("PCA Scores (PC1 vs PC2)")
```



```

Z = X_standard @ Vt.T

# Plot the PCA scores 3 and 4
sns.scatterplot(x=Z[:, 2], y=Z[:, 3], hue=df_iris['species'], ax=axes[2])
axes[2].set_title("PCA Scores (PC3 vs PC4)")

# print the loadings for the first two principal components
loadings = pd.DataFrame(Vt[:2, :], columns=feature_name, index=['PC1 (Vt[0]',
print(loadings.T)

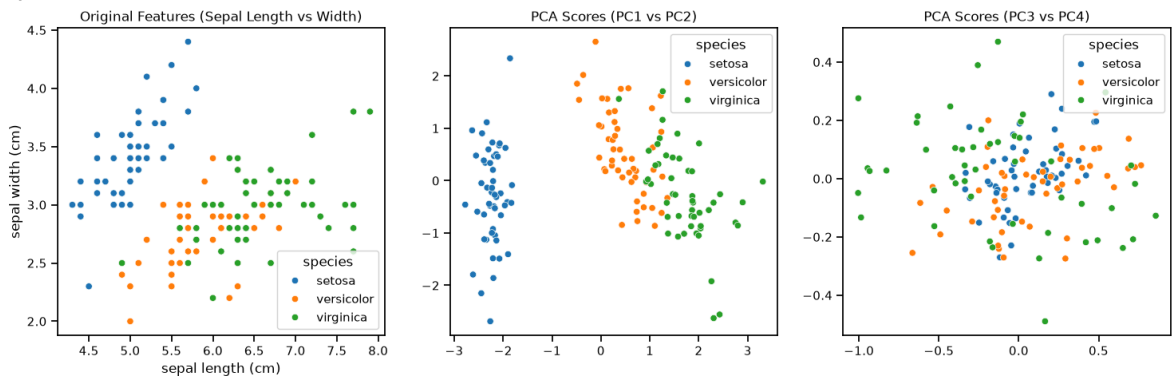
```

```

Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
      'petal width (cm)', 'species'],
      dtype='str')

```

	PC1 (Vt[0])	PC2 (Vt[1])
sepal length (cm)	0.521066	-0.377418
sepal width (cm)	-0.269347	-0.923296
petal length (cm)	0.580413	-0.024492
petal width (cm)	0.564857	-0.066942



Question: Which representation separates the species best?

Answer:

Based on the plots, PC1 vs PC2 separates the species best.

Question: Which variables make up the first component? Interpret the sign pattern of the second component.

Answer:

The first component (PC1) is primarily made up of petal length (+0.58), petal width (+0.57), and sepal length (+0.52), as these features have the highest absolute weight magnitudes in `Vt[0]`. Because all three carry large positive weights, PC1 acts as an index of overall flower size, meaning flowers with larger petals and longer sepals receive high positive PC1 scores. For the second component (PC2), sepal width (+0.92) is the dominant positive weight in `Vt[1]`, while all other features have negative weights (e.g., sepal length at (-0.38)). This opposing sign pattern creates a contrast/shape component that measures flower proportions rather than overall size. High positive PC2 scores represent flowers with unusually wide sepals relative to their length and petal size.

Reading the loadings. PC1 is a weighted sum of petal length, petal width and sepal length with the same sign, minus sepal width: "big flower with narrow sepals". PC2 is almost pure sepal width. The scores on PC3 and PC4 are small and show no species structure – that is the 4 % of variance we can drop.

The signs of singular vectors are **arbitrary** (flip u_j and v_j together and nothing changes). Do not over-interpret a sign, only sign *patterns* within one vector.

Part 3 – PCA as a rotation of the axes

To see what the first principal component *is*, we reduce the problem to two dimensions: petal length and petal width of the two similar species *versicolor* and *virginica*. In two dimensions a principal component is nothing more than a rotation of the coordinate system by an angle α :

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \alpha & \sin \alpha \\ -\sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

After the rotation, x' is the coordinate along the candidate component and y' is the perpendicular distance of each point from that axis, i.e. the part of the data that this component does **not** explain (z_i in ISL figure 12.2 and eq. 12.5). The first principal component is the angle for which $\sum_i y_i'^2$ is smallest.

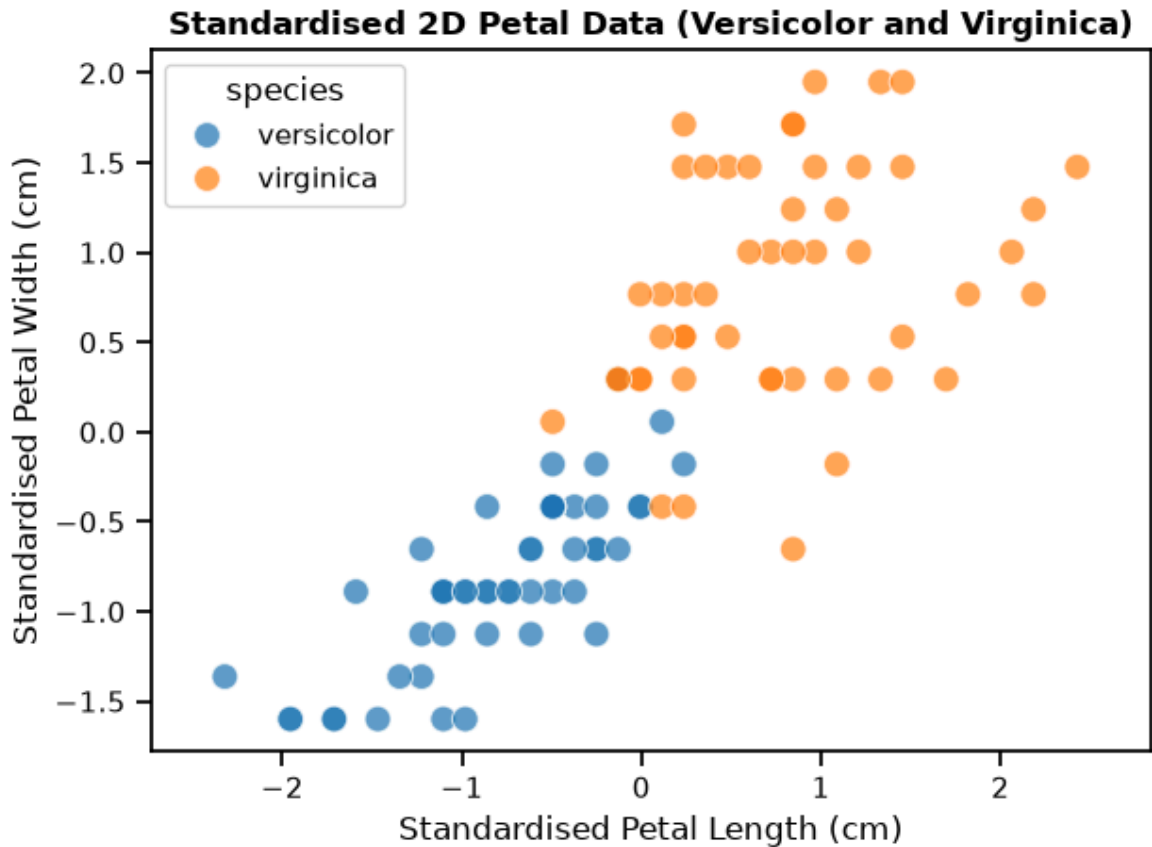
Task 3.1 – prepare the 2-D data

- Build a DataFrame with petal length and petal width for the varieties *versicolor* and *virginica* only.
- Centre each column (subtract the mean) and standardise (divide by the standard deviation).
- Scatter-plot the result with equal axis scaling.

```
In [27]: # your code here
# build a dataframe with the petal length and petal width for versicolor
df_iris_filtered = df_iris[df_iris['species'].isin(['versicolor', 'virginica'])]
df_iris_filtered = df_iris_filtered[['petal length (cm)', 'petal width (cm)']]
# df_iris_filtered['species'].value_counts()
```

```
In [40]: # Centre each column (subtract the mean) and standardise (divide by the standard deviation)
features = df_iris_filtered[['petal length (cm)', 'petal width (cm)']]
X_raw = features.values
# subtract the mean and divide by the standard deviation
X_std = (X_raw - X_raw.mean(axis=0)) / X_raw.std(axis=0)
# put standardised data into a dataframe for plotting
df_std = pd.DataFrame(X_std, columns=['petal length (cm)', 'petal width (cm)'])
df_std['species'] = df_iris_filtered['species'].values

# plot the standardised data
sns.scatterplot(data=df_std, x='petal length (cm)', y='petal width (cm)',
                plt.title("Standardised 2D Petal Data (Versicolor and Virginica)", fontweight='bold')
plt.xlabel("Standardised Petal Length (cm)")
plt.ylabel("Standardised Petal Width (cm)")
plt.axis("equal")
plt.tight_layout()
plt.show()
```



Task 3.2 – rotate

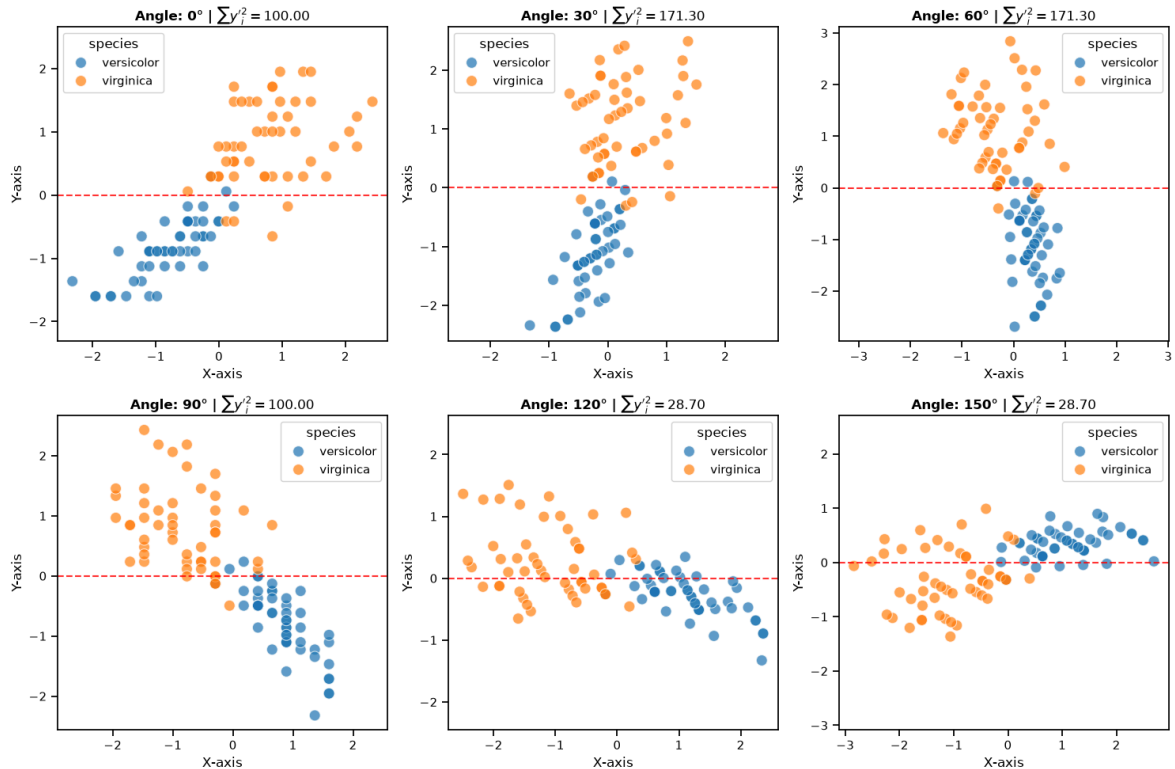
Write a function `rotate(xy, alpha_deg)` that applies the rotation matrix to an array of shape $(n, 2)$. Then make a composite figure (e.g. 2×4 panels) showing the rotated point cloud for angles between 0° and 180° . In each panel draw the new x' axis and print $\sum y_i'^2$ in the title.

```
In [52]: # your code here
def rotate_data(xy, alpha_deg):
    """
    Apply a rotation matrix to an array of shape (n, 2) and rotate it by
    """
    alpha_rad = np.radians(alpha_deg)
    rotation_matrix = np.array([[np.cos(alpha_rad), -np.sin(alpha_rad)],
                                [np.sin(alpha_rad), np.cos(alpha_rad)]])
    return xy @ rotation_matrix.T

# make a composite figure showing the rotated datapoint clouds for differ
angles = [0, 30, 60, 90, 120, 150]
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
for i, angle in enumerate(angles):
    ax = axes[i // 3, i % 3]
    rotated_data = rotate_data(X_std, angle)
    # calculate sum of squared y' values (unexplained variance along y')
    sum_y_sq = np.sum(rotated_data[:, 1] ** 2)
    sns.scatterplot(x=rotated_data[:, 0], y=rotated_data[:, 1], hue=df_ir)
    ax.axhline(0, color="red", linestyle="--", linewidth=1.5, alpha=0.8,
    ax.set_title(f"Angle: {angle}° |  $\sum y'_i{}^2 = {sum_y_sq:.2f}$ ", font
    ax.set_xlabel("X-axis")
    ax.set_ylabel("Y-axis")
```

```
ax.axis("equal")
plt.tight_layout()
plt.show()
```

```
<>:21: SyntaxWarning: invalid escape sequence '\s'
<>:21: SyntaxWarning: invalid escape sequence '\s'
/var/folders/7b/tyzwtcv13pz7k6db8rvxss_r0000gn/T/ipykernel_55034/116693770
2.py:21: SyntaxWarning: invalid escape sequence '\s'
ax.set_title(f"Angle: {angle}° |  $\sum y'_i{}^2 = \{sum\_y\_sq:.2f\}$ ", fontwe
ight='bold')
```



Task 3.3 – the unexplained variance as a function of the angle

For α from 0° to 360° compute $\sum_i y_i'^2$ and plot it against the angle. A polar plot makes the symmetry visible:

```
fig, ax = plt.subplots(subplot_kw={'projection': 'polar'})
ax.plot(alpha_rad, r)
```

Also plot $\sum_i x_i'^2$ (the explained part). What is $\sum x'^2 + \sum y'^2$, and why is it constant?

```
In [82]: # your code here
# Define smooth grid of angles from 0 to 360 degrees and convert to radians
alpha_deg = np.linspace(0, 360, 360)
alpha_rad = np.radians(alpha_deg)
sum_x_sq = []
sum_y_sq = []
# Calculate the sum of squared x' and y' coordinates for each angle of rotation
for angle in alpha_deg: # loop through each angle in degrees
    rotated = rotate_data(X_std, angle) # rotate the data by the current angle
    sum_x_sq.append(np.sum(rotated[:, 0]**2)) # Explained variance
    sum_y_sq.append(np.sum(rotated[:, 1]**2)) # Unexplained variance

# make lists into numpy arrays
```

```

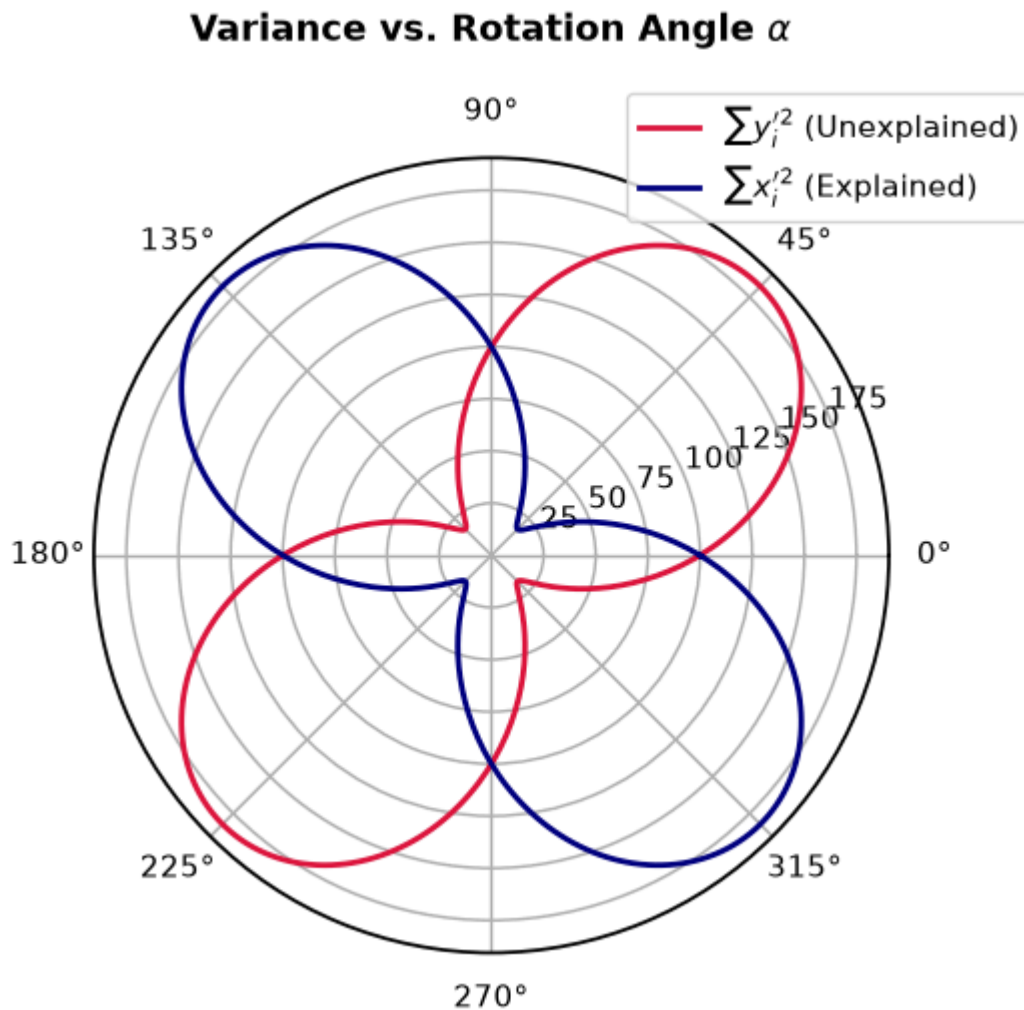
sum_x_sq = np.array(sum_x_sq)
sum_y_sq = np.array(sum_y_sq)

# now make a polar plot of the radius r as a function of angle alpha_rad
fig, ax = plt.subplots(subplot_kw={"projection": "polar"}, figsize=(8, 6))

ax.plot(alpha_rad, sum_y_sq, label="$\\sum y_i'^2$ (Unexplained)", color="red")
ax.plot(alpha_rad, sum_x_sq, label="$\\sum x_i'^2$ (Explained)", color="blue")
ax.set_title("Variance vs. Rotation Angle $\\alpha$", fontsize=13, fontweight="bold")
ax.legend(loc="upper right", bbox_to_anchor=(1.2, 1.1))

plt.tight_layout(pad=3.0)
plt.show()

```



Task 3.4 – compare with the SVD

Compute the SVD of the standardised 2-D data. The first right singular vector

`Vt[0]` points along the first principal component; its angle is

`np.arctan2(Vt[0,1], Vt[0,0])`. Compare with the minimum you found. Also compare s_2^2 with the minimum of the unexplained sum of squares, and s_1^2 with its maximum.

```

In [83]: # your code here
# compute the SVD of the standardised data
U, s, Vt = np.linalg.svd(X_std, full_matrices=False)
# Angle of the first principal component (Vt[0])

```

```

svd_angle_rad = np.arctan2(Vt[0, 1], Vt[0, 0]) # angle in radians
svd_angle_deg = np.degrees(svd_angle_rad) % 360 # convert to degrees
# Angle from rotation plot (min sum_y_sq or max sum_x_sq)
min_y_idx = np.argmin(sum_y_sq)
grid_angle_deg = alpha_deg[min_y_idx]
# Compare variances: singular values squared vs grid min/max
s1_sq = s[0] ** 2 # SVD max explained variance
s2_sq = s[1] ** 2 # SVD min unexplained variance
max_grid_x_sq = np.max(sum_x_sq) # Grid max explained variance
min_grid_y_sq = np.min(sum_y_sq) # Grid min unexplained variance

# Print results side-by-side
print("=== ANGLE COMPARISON ===")
print(f"Angle from SVD (Vt[0]): {svd_angle_deg:.2f}°")
print(f"Angle from Grid Search (min y'^2): {grid_angle_deg:.2f}°")

print("\n=== VARIANCE COMPARISON ===")
print(f"s1^2 (SVD Max Explained): {s1_sq:.4f} | Grid Max sum(x'^2): 182.3348")
print(f"s2^2 (SVD Min Unexplained): {s2_sq:.4f} | Grid Min sum(y'^2): 17.6660")

```

=== ANGLE COMPARISON ===

Angle from SVD (Vt[0]): 45.00°

Angle from Grid Search (min y'^2): 314.87°

=== VARIANCE COMPARISON ===

s1^2 (SVD Max Explained): 182.3348 | Grid Max sum(x'^2): 182.3340

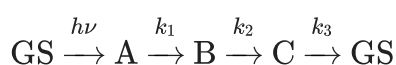
s2^2 (SVD Min Unexplained): 17.6652 | Grid Min sum(y'^2): 17.6660

Take-home. PCA finds the rotation that makes the *perpendicular* residuals smallest. That is different from linear regression, which minimises the *vertical* residuals (y given x). Regression treats one variable as the cause and the other as the effect; PCA treats them symmetrically. Because the two standardised variables here are strongly correlated, PC1 sits at $\approx 45^\circ$ – when all variables are standardised and equally correlated the first component is simply their average.

Part 4 – SVD on a spectro-kinetic dataset: what it can and cannot tell you

SVD is the workhorse for **time-resolved spectroscopy**: a transient-absorption matrix has time along one axis and wavelength along the other, and SVD tells you how many independent spectral components are needed to describe it. We construct such a dataset with **known** answers and try to retrieve them.

The model: a ground state GS is excited at $t = 0$ and relaxes through three intermediates



with lifetimes 0.1 ps, 10 ps and 1000 ps. Every species has a Gaussian spectrum. The measured signal is the **difference** absorbance, so the depleted ground state appears as a negative "bleach" with concentration $c_{\text{GS}}(t) = -(c_A + c_B + c_C)$. The data matrix is the sum of outer products

$$D(t, \lambda) = \sum_{\text{species } j} c_j(t) \varepsilon_j(\lambda) \quad \Longleftrightarrow \quad D = C E^T$$

(with C : time $\times$ species, E : wavelength $\times$ species). This is exactly the form SVD decomposes – so it should work perfectly, right?

The concentrations for a sequential first-order scheme have an analytic solution (the *Bateman* equations), which we use instead of a hand-written Euler loop; that keeps the example exact and fast.

```
In [85]: def sequential_kinetics(t, k1, k2, k3):
    '''Concentrations of A, B, C for A -> B -> C -> (GS) with A(0)=1 (Bat
    A = np.exp(-k1 * t)
    B = k1 / (k2 - k1) * (np.exp(-k1 * t) - np.exp(-k2 * t))
    C = k1 * k2 * (np.exp(-k1 * t) / ((k2 - k1) * (k3 - k1))
        + np.exp(-k2 * t) / ((k1 - k2) * (k3 - k2))
        + np.exp(-k3 * t) / ((k1 - k3) * (k2 - k3)))
    return np.column_stack([A, B, C])

    gauss = lambda x, mu, sigma=50: np.exp(-((x - mu) / sigma) ** 2)

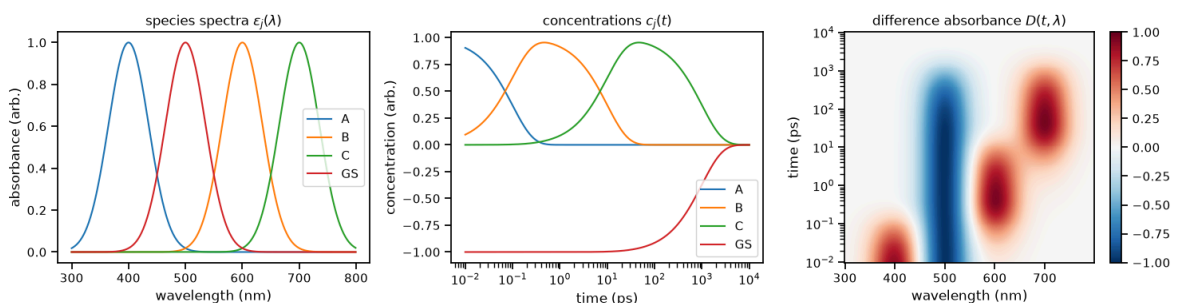
    wavelength = np.arange(300, 800, 1.0) # nm
    time = np.logspace(-2, 4, 300) # ps, log sp
    lifetimes = np.array([0.1, 10.0, 1000.0]) # ps
    rates = 1 / lifetimes

    spectra = pd.DataFrame({"A": gauss(wavelength, 400), "B": gauss(wavelength,
        "C": gauss(wavelength, 700), "GS": gauss(wavelength, 500)})
    spectra.index.name = "wavelength (nm)"

    conc = pd.DataFrame(sequential_kinetics(time, *rates), index=time, columns=["A", "B", "C", "GS"])
    conc["GS"] = -conc.sum(axis=1) # the bleach
    conc.index.name = "time (ps)"

    data = pd.DataFrame(conc.values @ spectra.values.T, index=conc.index, columns=spectra.columns)

    fig, ax = plt.subplots(1, 3, figsize=(15, 4))
    spectra.plot(ax=ax[0], title="species spectra $\varepsilon_j(\lambda)$")
    conc.plot(ax=ax[1], logx=True, title="concentrations $c_j(t)$"); ax[1].set(
    pm = ax[2].pcolormesh(data.columns, data.index, data.values, cmap="RdBu_r")
    ax[2].set(yscale="log", xlabel=spectra.index.name, ylabel=conc.index.name)
    fig.colorbar(pm, ax=ax[2]); plt.tight_layout(); plt.show()
    print("data shape (time x wavelength):", data.shape)
```



data shape (time x wavelength): (300, 500)

Task 4.1 – SVD of the data matrix

Apply `np.linalg.svd` to `data` (no centring here – why not?). Plot

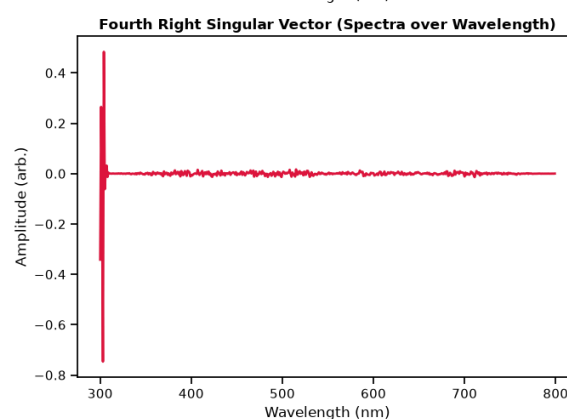
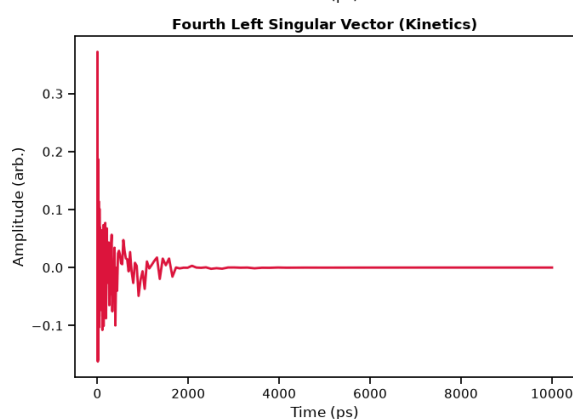
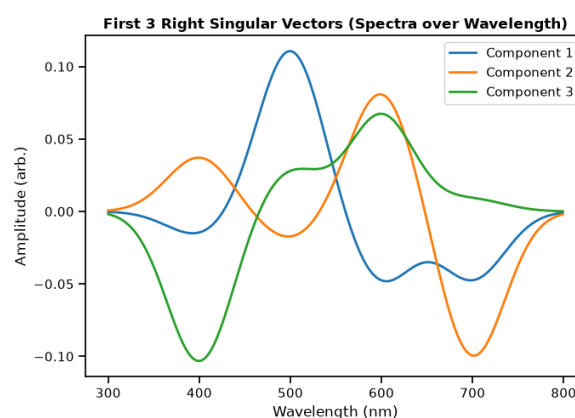
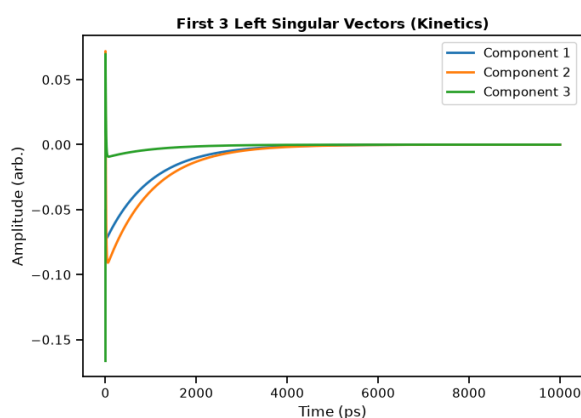
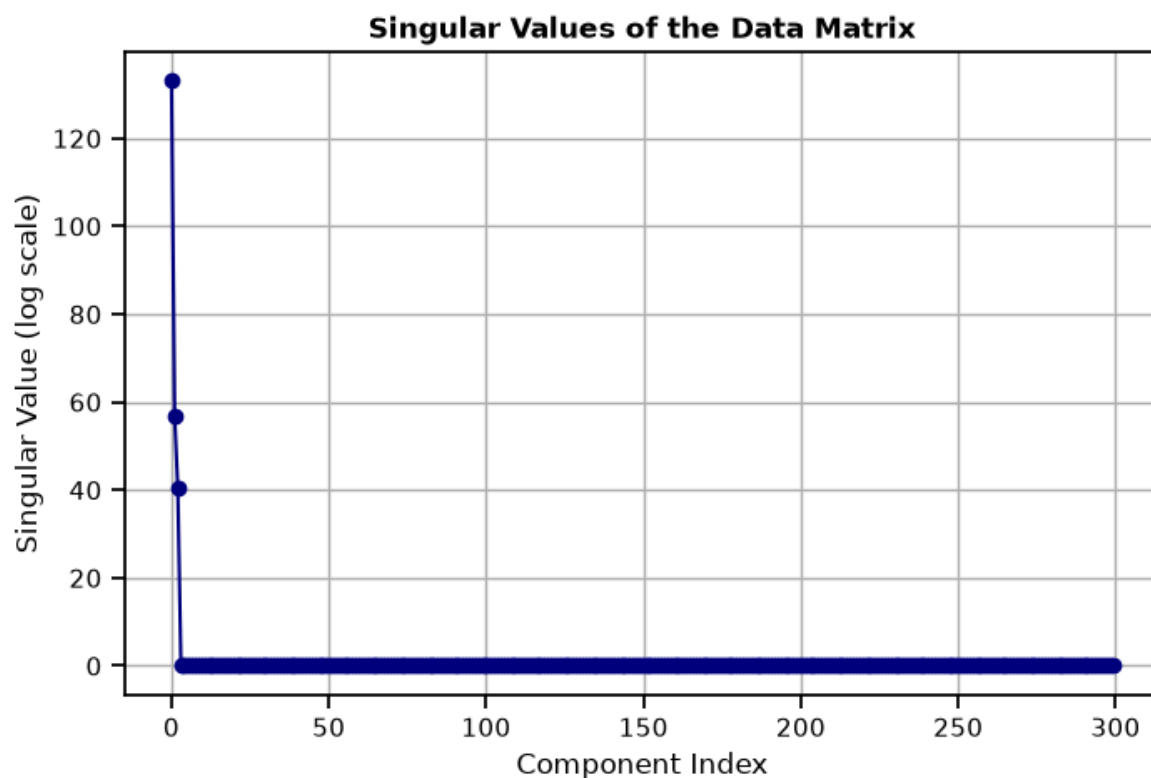
- the singular values (a scree plot, log scale),
- the first three **left** singular vectors against time (these are "kinetics"),
- the first three **right** singular vectors against wavelength (these are "spectra"),
- and, in a separate panel, the fourth pair – what does a singular vector that belongs to a singular value of 10^{-13} look like?

Check the shapes against the DataFrame first so you know which is which. **How many significant components are there? Why is that number different from the number of species? Do the singular vectors look like the spectra and concentrations we put in? Why not?**

```
In [99]: # your code here
# compute SVD without centering
U, s, Vt = np.linalg.svd(data.values, full_matrices=False)
# plot the singular values
plt.figure(figsize=(8, 5))
plt.plot(s, marker='o', color='navy')
plt.title("Singular Values of the Data Matrix", fontweight='bold')
plt.xlabel("Component Index")
plt.ylabel("Singular Value (log scale)")
plt.grid()

# First 3 Left Singular Vectors (Kinetics over Time)
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
for i in range(3):
    axes[0, 0].plot(data.index, U[:, i], lw=2, label=f"Component {i+1}")
axes[0, 0].set_title("First 3 Left Singular Vectors (Kinetics)", fontweight='bold')
axes[0, 0].set_xlabel("Time (ps)")
axes[0, 0].set_ylabel("Amplitude (arb.)")
axes[0, 0].legend()
# First 3 Right Singular Vectors (Spectra over Wavelength)
for i in range(3):
    axes[0, 1].plot(data.columns, Vt[i, :], lw=2, label=f"Component {i+1}")
axes[0, 1].set_title("First 3 Right Singular Vectors (Spectra over Wavelength)", fontweight='bold')
axes[0, 1].set_xlabel("Wavelength (nm)")
axes[0, 1].set_ylabel("Amplitude (arb.)")
axes[0, 1].legend()
# Fourth Left Singular Vector (Kinetics over Time)
axes[1, 0].plot(data.index, U[:, 3], lw=2, color='crimson', label="Component 4")
axes[1, 0].set_title("Fourth Left Singular Vector (Kinetics)", fontweight='bold')
axes[1, 0].set_xlabel("Time (ps)")
axes[1, 0].set_ylabel("Amplitude (arb.)")
# Fourth Right Singular Vector (Spectra over Wavelength)
axes[1, 1].plot(data.columns, Vt[3, :], lw=2, color='crimson', label="Component 4")
axes[1, 1].set_title("Fourth Right Singular Vector (Spectra over Wavelength)", fontweight='bold')
axes[1, 1].set_xlabel("Wavelength (nm)")
axes[1, 1].set_ylabel("Amplitude (arb.)")

plt.tight_layout()
plt.show()
```

Answer. Only **three** singular values are non-zero, although we built the data from **four** species. The reason is the mass balance $c_{GS} = -(c_A + c_B + c_C)$: the concentration matrix has rank 3, so the data matrix cannot have rank 4. SVD counts *linearly independent* components, not chemical species – a difference spectroscopy experiment can never tell you more than $n_{\text{species}} - 1$ from the rank alone.

The singular vectors are also **not** the spectra or kinetics we put in. Each v_j is a mixture of several Gaussians with positive and negative lobes, and the u_j show unphysical sign changes. SVD returns an *orthonormal* basis ordered by variance; real

spectra are neither orthogonal nor ordered. Any invertible matrix T gives an equally valid factorisation $D = (CT)(T^{-1}E^T)$ – this is the **rotational ambiguity**. SVD alone therefore reduces dimensions and denoises, but the *interpretation* needs additional information: a kinetic model, non-negativity, known spectra ... which is what "global analysis" (part 4.3) adds.

Task 4.2 – low-rank approximation and noise

Add Gaussian noise (e.g. standard deviation 0.05) to `data`, repeat the SVD and look at the scree plot again. Reconstruct the data from the first k components only, $D_k = U_{:,k} \text{diag}(s_{:k}) V_{:,k}^T$, for $k = 1, 2, 3, 6$ and plot the residual $D_{\text{noisy}} - D_k$ (as an image) and its standard deviation. What is the "right" k and how do you see it?

```
In [105... # your code here
# svd on the not noisy data
U, s, Vt = np.linalg.svd(data.values, full_matrices=False)
sigma_noise = 0.05
np.random.seed(42)
data_noisy = data + np.random.normal(0, sigma_noise, size=data.shape)
# compute SVD on noise data
U_n, s_n, Vt_n = np.linalg.svd(data_noisy.values, full_matrices=False)
# compare the noise free data and the noisy data
plt.figure(figsize=(7, 4))
plt.semilogy(range(1, len(s) + 1), s, "o--", label="Clean Data", alpha=0.5)
plt.semilogy(range(1, len(s_n) + 1), s_n, "o-", label="Noisy Data", color="red")
plt.axhline(sigma_noise * np.sqrt(max(data.shape)), color="gray", linestyle="dashed")
plt.title("Scree Plot: Clean vs. Noisy Data", fontweight="bold")
plt.xlabel("Component Index")
plt.ylabel("Singular Value (log scale)")
plt.legend()
plt.grid(True, which="both", linestyle="--", alpha=0.5)
plt.show()

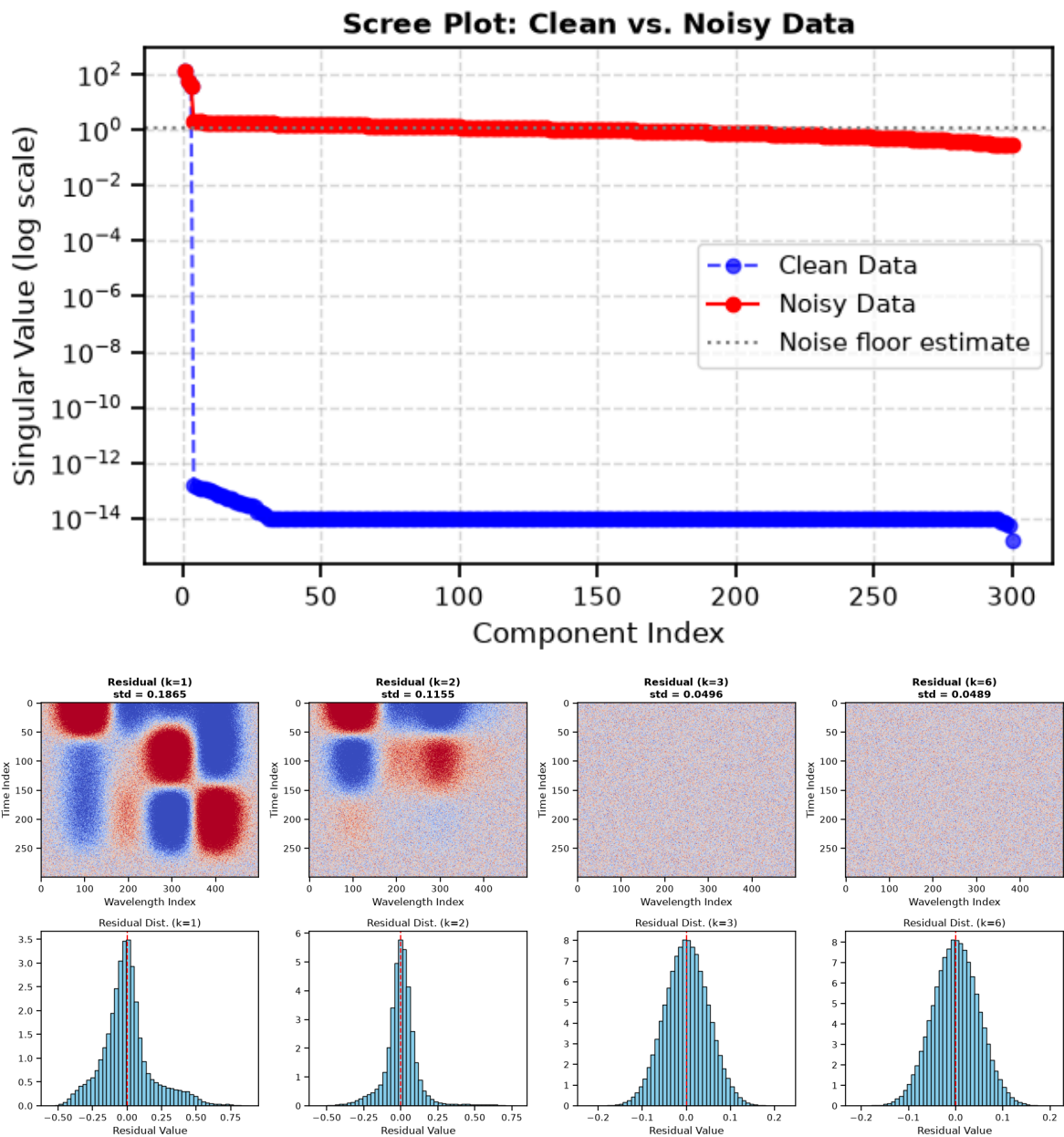
# low-rank reconstruction and residual analysis
k_values = [1, 2, 3, 6]
fig, axes = plt.subplots(2, 4, figsize=(18, 8))

for idx, k in enumerate(k_values):
    # truncate to first k components for the noisy data
    U_k = U_n[:, :k]
    S_k = s_n[:k]
    Vt_k = Vt_n[:, :k]
    # low-rank approximation D_k = U_k * diag(S_k) * Vt_k
    D_k = U_k @ np.diag(S_k) @ Vt_k
    # compute residual
    residual = data_noisy - D_k
    res_std = np.std(residual)
    # plot residual matrix as heatmap/image
    im = axes[0, idx].imshow(residual, aspect="auto", cmap="coolwarm", vmin=0, vmax=1)
    axes[0, idx].set_title(f"Residual (k={k})\nstd = {res_std:.4f}", fontweight="bold")
    axes[0, idx].set_xlabel("Wavelength Index")
    axes[0, idx].set_ylabel("Time Index")

    # Plot Histogram/Distribution of Residuals
    axes[1, idx].hist(residual.to_numpy().ravel(), bins=50, density=True,
```

```
axes[1, idx].axvline(0, color="red", linestyle="--", label="Zero line")
axes[1, idx].set_title(f"Residual Dist. (k={k})")
axes[1, idx].set_xlabel("Residual Value")
```

```
plt.tight_layout()
plt.show()
```



Answer. With noise, the scree plot shows three singular values standing clearly above a flat "noise floor"; the floor is where the singular values of pure noise sit. With $k = 1$ and $k = 2$ the residual still contains structure (you can see the missing species); at $k = 3$ the residual is featureless and its standard deviation equals the noise we added (0.05). Adding more components only fits noise – $k = 6$ lowers the residual a little, but by modelling noise, not chemistry. The truncated SVD D_3 is a **denoised** version of the data; this is the "low-rank approximation" from the lecture.

Task 4.3 – from SVD to global analysis

SVD told us *how many* components there are. To get physically meaningful spectra we add a **model**: we know (or guess) that the kinetics are sequential with three lifetimes. Then for a given set of lifetimes the concentration matrix $C(t)$ is fixed and the spectra follow from a linear least-squares problem

$$E^T = \arg \min_E \|D - CE^T\|^2 \Rightarrow E^T = C^+ D$$

(`np.linalg.lstsq(C, D)`). The lifetimes themselves are then found by non-linear least squares on the residual (`scipy.optimize.least_squares`) – this is what packages such as *Glutaran* or *KiMoPack* do.

One subtlety follows directly from the rank argument above: because $c_{GS} = -(c_A + c_B + c_C)$, the four-column concentration matrix is singular, and `lstsq` would return an arbitrary (minimum-norm) split between species and bleach. So we fit with the **three** independent columns c_A, c_B, c_C only. The spectra we then obtain are the **species-associated difference spectra** $\Delta\epsilon_j = \epsilon_j - \epsilon_{GS}$ – every species is seen relative to the ground state it came from. This is the standard result of a transient-absorption analysis.

- Write a function `residual(log_tau)` that builds C (three columns) for the three lifetimes (in log scale so they stay positive), solves for E by `lstsq`, and returns the flattened residual $D_{\text{noisy}} - CE^T$.
- Start from deliberately wrong guesses (e.g. lifetimes 0.3, 30, 300 ps), run `least_squares`, and compare the fitted lifetimes and difference spectra with the truth ($\epsilon_j - \epsilon_{GS}$).

```
In [114... # your code here
def residual(log_tau, times, data_matrix):
    """
    Computes the flattened residual (data_matrix - D_model) for scipy.optimize.least_squares.
    Parameters:
    * log_tau: logarithm of lifetimes in picoseconds.
    * times: time points corresponding to rows of data_matrix.
    * data_matrix: noisy dataset
    """
    # convert tau back: e^log_tau = tau
    tau = np.exp(log_tau)
    # convert lifetimes to rate constants: k = 1 / tau
    k1, k2, k3 = 1.0 / tau
    # Build concentration matrix C using Bateman equation
    C = sequential_kinetics(times, k1, k2, k3)
    # solve for E
    E = np.linalg.lstsq(C, data_matrix, rcond=None)[0] # slices-extract t
    # calculate the reconstructed model data
    model = C @ E
    # return the flattened difference vector for scipy.optimize.least_squares
    return (model - data_matrix).ravel()
```

```
In [115... times = data.index.to_numpy()
# start with the setting up wrong guesses:
tau_initial = np.array([0.3, 30.0, 300.0])
log_tau_initial = np.log(tau_initial)
# extract values array if data_noisy is a DataFrame
```

```

data_array = (data_noisy.values if hasattr(data_noisy, "values") else data_noisy)
times = data.index.to_numpy() if hasattr(data, "index") else data.index
opt_result = least_squares(residual, log_tau_initial, args=(times, data_array))
tau_fitted = np.exp(opt_result.x)

print("=== LIFETIME FITTING RESULTS ===")
print(f"Initial Guesses : {tau_initial} ps")
print(f"True Lifetimes : [0.1, 10.0, 1000.0] ps")
print(f"Fitted Lifetimes: {np.round(tau_fitted, 2)} ps")

```

```

=== LIFETIME FITTING RESULTS ===
Initial Guesses : [ 0.3 30. 300. ] ps
True Lifetimes : [0.1, 10.0, 1000.0] ps
Fitted Lifetimes: [1.00000e-01 1.00200e+01 1.00083e+03] ps

```

Take-home for the spectroscopist. SVD is the model-free first look: it tells you the rank (how many components) and denoises. It does *not* give you species spectra, because of the rotational ambiguity and because the bleach couples the species. Even with a model, the linear dependence between bleach and species survives: you get *difference* spectra, and recovering the absolute ε_j needs one more piece of information (the ground-state spectrum, measured separately). Global analysis breaks the ambiguity by imposing a kinetic model – then the linear part (spectra) is solved exactly by least squares and only the few non-linear parameters (lifetimes) need iteration. The same structure – linear parameters solved by `lstsq`, non-linear ones by an optimiser – is called *variable projection* and is used far beyond spectroscopy.

Part 5 – Clustering with k-means

We return to the iris data. **k-means** minimises the within-cluster sum of squares

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

by alternating between assigning each point to the nearest centroid and moving each centroid to the mean of its points. It always converges, but only to a *local* minimum – the result depends on the random start. scikit-learn therefore runs it `n_init` times and keeps the best.

Task 5.1 – k and the random start

Use the code skeleton below (5 × 5 panels; the first row shows the true species in the petal-length/petal-width plane). Fill rows 2–5 with `sklearn.cluster.KMeans` results on **all four** columns: across the columns use `n_clusters = 1 ... 5`, across the rows use `n_init = 1, 2, 5, 20` (with `random_state=2`, or vary it). Look closely: where and when do the results differ? Then repeat with only two columns (e.g. sepal length and sepal width) as input and describe the difference.

```
In [116... def kmeans_grid(X, title):
    '''5x5 grid: true labels in row 0, k-means with k=1..5 (columns) and
    fig, ax = plt.subplots(5, 5, figsize=(12, 12), sharex=True, sharey=True)
    for sp in iris.target_names:
        m = (species == sp).values
        for j in range(5):
            ax[0, j].scatter(df_iris.iloc[m, 2], df_iris.iloc[m, 3], s=4,
            ax[0, 0].legend(fontsize=7, markerscale=3); ax[0, 0].set_ylabel("true
            # --- rows 1..4 are filled in Task 5.1 ---
        return fig, ax

fig, ax = kmeans_grid(df_iris.values, "all four columns")
plt.close(fig) # only a skeleton; the real figure is produced below
```

```
In [134... print(df_iris.columns.tolist()) # overview of the Iris data set

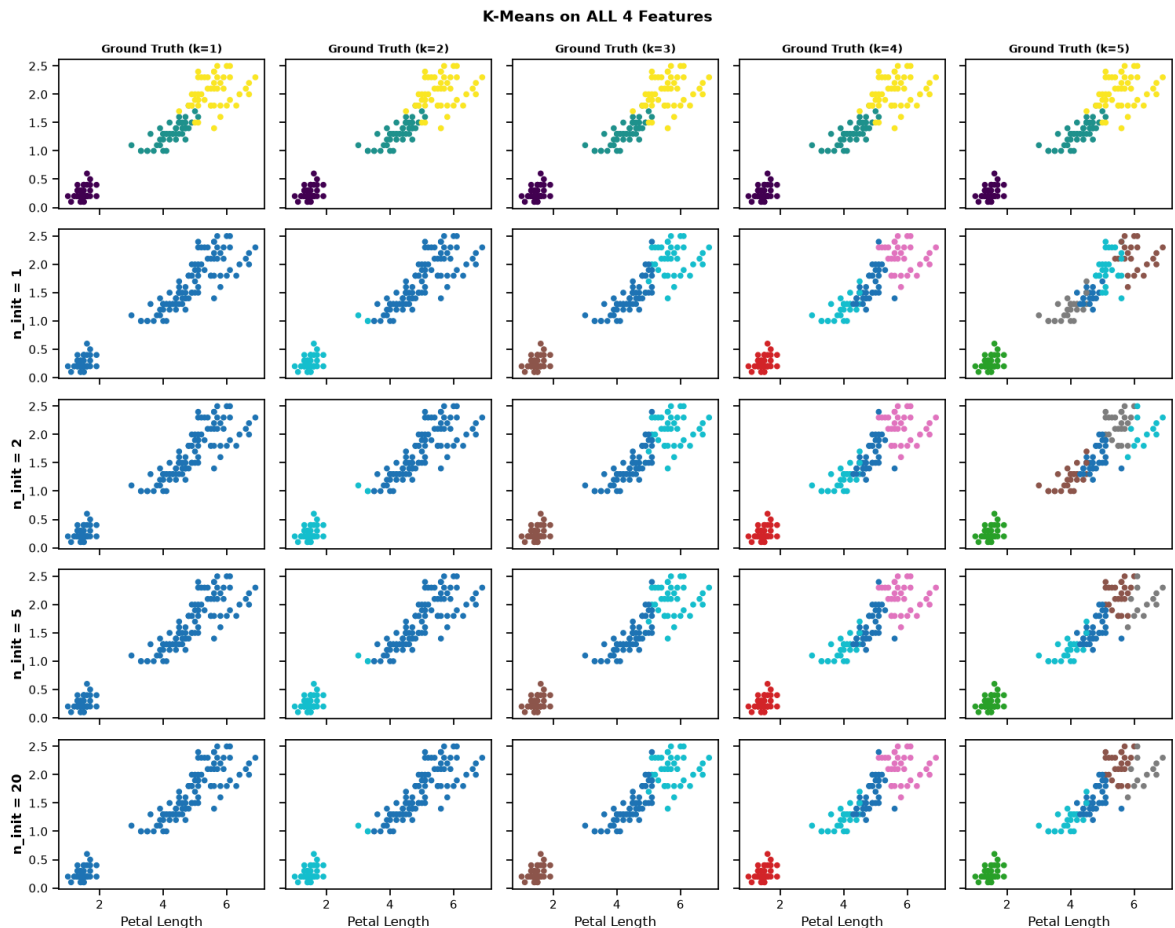
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width
h (cm)', 'species']
```

```
In [ ]: # your code here
# load data features
features_all = df_iris.iloc[:, :4] # All 4 features
features_2D = df_iris.iloc[:, :2] # First 2 features (sepal length & width)
x_petal = df_iris.iloc[:, 2] # Column index 2: Petal Length
y_petal = df_iris.iloc[:, 3] # Column index 3: Petal Width
# species data
species_codes = pd.factorize(df_iris.iloc[:, -1])[0]
# parameter ranges
n_inits = [1, 2, 5, 20]
k_values = [1, 2, 3, 4, 5]

fig, axes = plt.subplots(5, 5, figsize=(15, 12), sharex=True, sharey=True)
# for row 0 (True Species Ground Truth)
for j, k in enumerate(k_values):
    axes[0, j].scatter(x_petal, y_petal, c=species_codes, cmap="viridis",
    axes[0, j].set_title(f"Ground Truth (k={k})", fontsize=10, fontweight="bold")
# for rows 1 to 4 (KMeans with varying n_init and k)
for i, n_init in enumerate(n_inits):
    for j, k in enumerate(k_values):
        # Fit KMeans on all 4 features
        kmeans = KMeans(n_clusters=k, n_init=n_init, random_state=2, max_iter=1000)
        labels = kmeans.fit_predict(features_all)

        # Scatter plot in Petal Length vs. Petal Width space
        axes[i + 1, j].scatter(x_petal, y_petal, c=labels, cmap="tab10",
        if j == 0: axes[i + 1, j].set_ylabel(f"n_init = {n_init}", fontweight="bold")
        if i == 3: axes[i + 1, j].set_xlabel("Petal Length")

plt.suptitle("K-Means on ALL 4 Features", fontsize=14, fontweight="bold",
plt.tight_layout()
plt.show()
```

What to notice. With $k = 3$ on all four columns k-means recovers *setosa* perfectly and mixes *versicolor/virginica* only at their border. The inertia J always falls when k rises – so J alone cannot pick k . With `n_init=1` and an unlucky seed the $k = 4$ or 5 solutions differ from run to run (compare the J values): that is the local-minimum problem. With only sepal length and width as input, the algorithm must draw its borders in a plane where the species overlap, and the clusters no longer follow the species in the petal plane at all: **clusters are only as good as the features you feed in.**

Task 5.2 – how many clusters? Elbow and silhouette

Two standard criteria:

- **Elbow:** plot the inertia $J(k)$ for $k = 1 \dots 10$ and look for the bend.
- **Silhouette:** for each point i , with a_i the mean distance to the other points in its own cluster and b_i the mean distance to the points of the nearest *other* cluster,

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \in [-1, 1].$$

The mean silhouette over all points (`sklearn.metrics.silhouette_score`) is large when clusters are compact and well separated; scan $k = 2 \dots 10$ and pick the maximum (or the smallest k on a plateau).

Compute both for the **standardised** iris data and for the raw data. Do they agree with the three species? Also compute the `adjusted_rand_score` between the $k = 3$ labels and the true species (a chance-corrected agreement score, 1 = perfect) – for us the only "ground truth" check we have.

```
In [143... # your code here
from sklearn.metrics import adjusted_rand_score, silhouette_score
from sklearn.preprocessing import StandardScaler
# load data features
features_raw = df_iris.iloc[:, :4].values
features_std = StandardScaler().fit_transform(features_raw)
# species data
true_species = pd.factorize(df_iris.iloc[:, -1])[0]
# compute Inertia and Silhouette Scores for k = 1..10
k_range = range(1, 11)
inertia_raw, inertia_std = [], []
sil_raw, sil_std = [], []

for k in k_range:
    km_raw = KMeans(n_clusters=k, n_init=20, random_state=42).fit(features_raw)
    inertia_raw.append(km_raw.inertia_)
    if k > 1:
        sil_raw.append(silhouette_score(features_raw, km_raw.labels_))
    km_std = KMeans(n_clusters=k, n_init=20, random_state=42).fit(features_std)
    inertia_std.append(km_std.inertia_)
    if k > 1:
        sil_std.append(silhouette_score(features_std, km_std.labels_))

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
# plot elbow curves (Inertia)
axes[0].plot(k_range, inertia_raw, "o-", label="Raw Data", color="tab:blue")
axes[0].plot(k_range, inertia_std, "s--", label="Standardized Data", color="tab:red")
axes[0].axvline(x=3, color="gray", linestyle=":", label="k = 3 (True species)")
axes[0].set_title("Elbow Method (Inertia vs. k)", fontweight="bold")
axes[0].set_xlabel("Number of Clusters (k)")
axes[0].set_ylabel("Inertia / Within-Cluster SS ($J$)")
axes[0].set_xticks(k_range)
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# plot silhouette scores
axes[1].plot(range(2, 11), sil_raw, "o-", label="Raw Data", color="tab:blue")
axes[1].plot(range(2, 11), sil_std, "s--", label="Standardized Data", color="tab:red")
axes[1].axvline(x=3, color="gray", linestyle=":", label="k = 3 (True species)")
axes[1].set_title("Silhouette Score vs. k", fontweight="bold")
axes[1].set_xlabel("Number of Clusters (k)")
axes[1].set_ylabel("Mean Silhouette Score")
axes[1].set_xticks(k_range[1:])
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

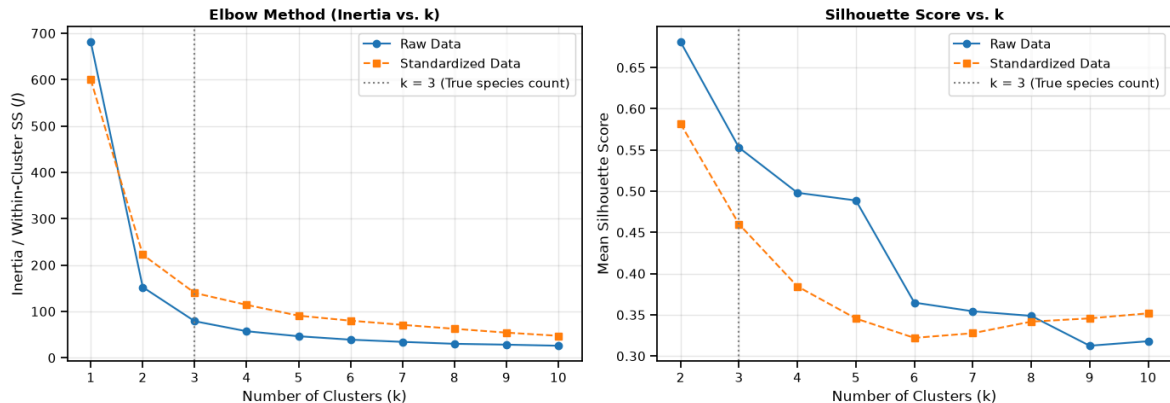
# 4. Compute Adjusted Rand Score (ARI) for k = 3
km3_raw = KMeans(n_clusters=3, n_init=20, random_state=42).fit_predict(features_raw)
km3_std = KMeans(n_clusters=3, n_init=20, random_state=42).fit_predict(features_std)
```

```

ari_raw = adjusted_rand_score(true_species, km3_raw)
ari_std = adjusted_rand_score(true_species, km3_std)

print("=== ADJUSTED RAND SCORE (ARI) FOR k = 3 ===")
print(f"Raw Data ARI          : {ari_raw:.4f}")
print(f"Standardized Data ARI   : {ari_std:.4f}")

```



```

=== ADJUSTED RAND SCORE (ARI) FOR k = 3 ===
Raw Data ARI          : 0.7302
Standardized Data ARI   : 0.6201

```

Discussion. Both criteria prefer **k = 2**: *setosa* is far away from the other two species, which overlap. Three is defensible as a second choice, especially in the raw data. This is the honest answer of unsupervised learning: the data contain two clearly separated groups, and the split of the second group into two species is information that only the labels carry. Note that k-means on the raw data actually agrees *better* with the species (higher ARI) than on standardised data, because scaling inflates the noisy sepal width. Scaling is not automatically better.

A silhouette **plot** (per-point silhouettes sorted within each cluster, `sklearn.metrics.silhouette_samples`) shows *which* cluster is weak; the mean alone hides this.

Part 6 – Hierarchical clustering

Agglomerative clustering starts with every point as its own cluster and merges the two closest clusters until one remains. The whole history of merges is the **dendrogram**; cutting it at a height gives a partition. "Closest" depends on the **linkage**:

linkage	distance between clusters	tends to give
single	min over all pairs	long chains, sensitive to noise bridges
complete	max over all pairs	compact, similar-sized clusters
average	mean over all pairs	in between
Ward	increase of within-cluster variance on merging	compact, like k-means

Task 6.1 – dendrograms

Use `scipy.cluster.hierarchy.linkage` on the standardised iris data with the four linkage methods and draw the four dendrograms (`scipy.cluster.hierarchy.dendrogram`, colour the leaves by the true species with `link_color_func` or simply compare the leaf order). Where would you cut each tree? Then cut at three clusters (`fcluster(..., 3, 'maxclust')`, or `sklearn.cluster.AgglomerativeClustering`) and compute the adjusted Rand index against the species.

```
In [144... # your code here
from scipy.cluster.hierarchy import cut_tree, dendrogram, fcluster, linkage
# load data features
features_raw = df_iris.iloc[:, :4].values
features_std = StandardScaler().fit_transform(features_raw)
# species data
true_species = pd.factorize(df_iris.iloc[:, -1])[0]
# define linkage methods
linkage_methods = ["single", "complete", "average", "ward"]
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes = axes.ravel()
# store results
ari_results = {}

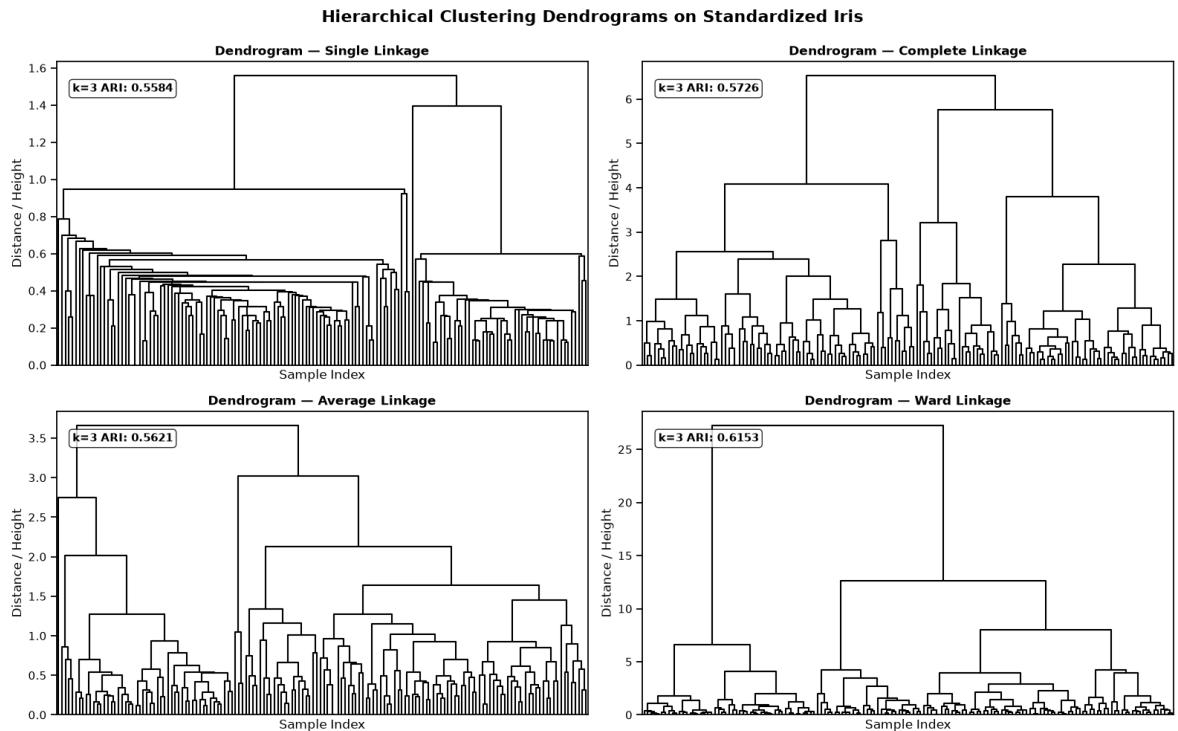
for idx, method in enumerate(linkage_methods):
    # compute hierarchical linkage matrix using SciPy
    Z = linkage(features_std, method=method)

    # plot dendrogram
    dendrogram(Z, ax=axes[idx], no_labels=True, color_threshold=0, above_

    axes[idx].set_title(f"Dendrogram – {method.capitalize()} Linkage", fo
    axes[idx].set_ylabel("Distance / Height")
    axes[idx].set_xlabel("Sample Index")
    # cut tree at k = 3 clusters using fcluster
    clusters = fcluster(Z, t=3, criterion="maxclust") - 1 # 0-indexed
    # calculate Adjusted Rand Index (ARI)
    ari = adjusted_rand_score(true_species, clusters)
    ari_results[method] = ari
    # annotate plot with ARI
    axes[idx].text(0.03, 0.90, f"k=3 ARI: {ari:.4f}", transform=axes[idx]

plt.suptitle("Hierarchical Clustering Dendrograms on Standardized Iris",
plt.tight_layout()
plt.show()

# Print ARI Summary
print("=== ADJUSTED RAND INDEX (ARI) FOR k = 3 ===")
for method, score in ari_results.items():
    print(f"{method.capitalize():<10} Linkage ARI : {score:.4f}")
```



=== ADJUSTED RAND INDEX (ARI) FOR k = 3 ===

Single	Linkage ARI : 0.5584
Complete	Linkage ARI : 0.5726
Average	Linkage ARI : 0.5621
Ward	Linkage ARI : 0.6153

Discussion. Ward and complete linkage give the "textbook" trees with one big early split (setosa vs. the rest) and a defensible second split. Single linkage chains the overlapping *versicolor/virginica* points into one cluster and peels off outliers one by one – the classic failure mode of single linkage on data without gaps. The dendrogram's vertical axis answers the question "how many clusters?" visually: a long vertical stretch without merges means the clusters below it are well separated. Cutting there is the hierarchical equivalent of the elbow.

Part 7 – Choosing the right clustering method: the smileys

The folder with the smiley images contains point clouds: a clean set (`cl_*`) and a noisy set (`bl_*`). Every dark pixel is one observation with two features, its x and y position. The task is to recover the "domains" (eyes, mouth, ring). This mimics a very common problem in microscopy – e.g. the magnetic domains in the NdFeB micrograph in the same folder (Gorchy, CC BY-SA 3.0, [Wikimedia Commons](#)).

We first turn an image into a point cloud with scikit-image (no OpenCV needed).

```
In [159... import skimage.transform
from sklearn.neighbors import NearestNeighbors

def image_to_points(path, threshold=0.5, binning="auto", max_points=4000,
    # Turn an image into a point cloud: every dark pixel becomes one obse
    # binning > 1 first averages blocks of binning x binning pixels (skim
```

```

# which raises the signal-to-noise ratio of a dotted image before thr
# bin size that brings every image to about target_width pixels, so a
# Coordinates are divided by the image width, so all images live in t
# a distance parameter such as DBSCAN eps has the same meaning for ev
rng = rng or np.random.default_rng(RNG_SEED)
img = skimage.io.imread(path)
if img.ndim == 3 and img.shape[2] == 4:                                # RGBA: p
    alpha = img[..., 3:4] / 255.0
    img = (img[..., :3] * alpha + 255 * (1 - alpha)).astype(np.uint8)
gray = skimage.color.rgb2gray(img) if img.ndim == 3 else img / 255.0
if binning == "auto":
    binning = max(1, round(gray.shape[1] / target_width))
if binning > 1:
    h, w = (np.array(gray.shape) // binning) * binning                # crop to
    gray = skimage.transform.downscale_local_mean(gray[:h, :w], (binn
ys, xs = np.nonzero(gray < threshold)                                # dark p
pts = np.column_stack([xs, -ys]).astype(float) / gray.shape[1]        # fl
if len(pts) > max_points:                                             # subsam
    pts = pts[rng.choice(len(pts), max_points, replace=False)]
return pts

def typical_spacing(pts):
    # median distance from a point to its nearest neighbour - a natural l
    d, _ = NearestNeighbors(n_neighbors=2).fit(pts).kneighbors(pts)
    return np.median(d[:, 1])

def plot_clusters(ax, pts, labels, title):
    noise = labels == -1
    ax.scatter(pts[~noise, 0], pts[~noise, 1], c=labels[~noise], cmap="ta
    if noise.any(): ax.scatter(pts[noise, 0], pts[noise, 1], c="lightgrey
    ax.set(title=title, aspect="equal", xticks=[], yticks=[])

def n_clusters(labels):
    return len(set(labels)) - (1 if -1 in labels else 0)

smiley_files = sorted(SMILEY_DIR.glob("*.png")) if SMILEY_DIR else []
clean = [f for f in smiley_files if f.stem.startswith("cl")]
noisy = [f for f in smiley_files if f.stem.startswith("bl")]
print([f.name for f in smiley_files])

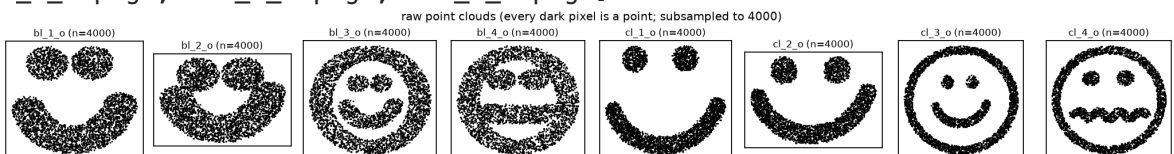
fig, ax = plt.subplots(1, len(smiley_files), figsize=(2.6 * len(smiley_fi
for a, f in zip(np.atleast_1d(ax), smiley_files):
    pts = image_to_points(f, binning=1)
    a.scatter(pts[:, 0], pts[:, 1], s=1, c="k"); a.set(title=f"{f.stem} (
fig.suptitle("raw point clouds (every dark pixel is a point; subsampled t

```

```

['bl_1_o.png', 'bl_2_o.png', 'bl_3_o.png', 'bl_4_o.png', 'cl_1_o.png', 'cl
_2_o.png', 'cl_3_o.png', 'cl_4_o.png']

```



Task 7.1 – k-means vs DBSCAN vs spectral clustering

For each **clean** smiley (use the default `binning="auto"`, which brings every image to about 160 pixels width so that the clouds are dense, uniform and quick to handle) cluster the point cloud with

- `sklearn.cluster.KMeans` (choose `k` = number of domains you see),
- `sklearn.cluster.DBSCAN` – a density-based method: a point is a *core point* if at least `min_samples` points lie within distance `eps`; connected core points form a cluster; the rest is noise (label `-1`). It needs no `k`, but it needs `eps`. A good starting point is a few times the typical nearest-neighbour spacing (`typical_spacing`).
- `sklearn.cluster.SpectralClustering` (with `affinity='nearest_neighbors'`) – builds a neighbourhood graph and clusters its eigenvectors; it can follow curved shapes.
- `sklearn.cluster.AgglomerativeClustering(linkage='single')` – single linkage should also follow connected shapes.

Show the results side by side. Which methods separate the ring from the face? Why does k-means fail there?

In [158...

```
# your code here

from sklearn.cluster import (DBSCAN, AgglomerativeClustering, SpectralClustering)
from sklearn.neighbors import NearestNeighbors

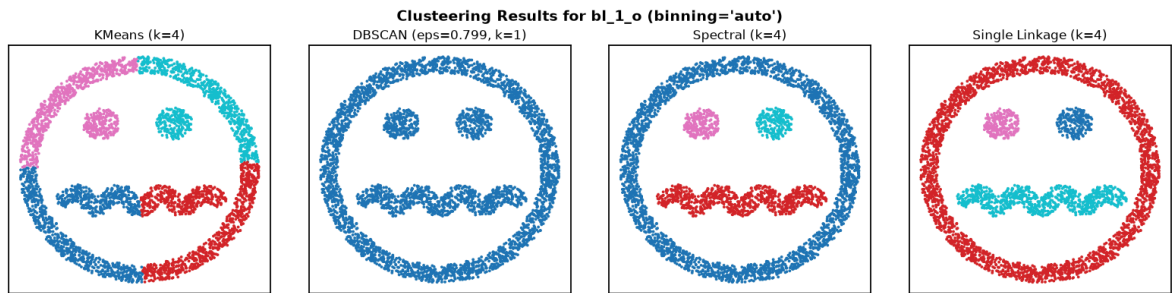
# estimate spacing to set eps for DBSCAN
nbrs = NearestNeighbors(n_neighbors=2).fit(X)
distances = nbrs.kneighbors(X)[0]
spacing = np.median(distances[:, 1])
eps_guess = 3.0 * spacing

# number of domains for k-means / spectral (e.g., 1 ring + 2 eyes + 1 mouth)
n_domains = 4

# fit all 4
# KMeans
kmeans_labels = KMeans(n_clusters=n_domains, n_init=10, random_state=42).fit(X).labels_
# DBSCAN
dbscan = DBSCAN(eps=eps_guess, min_samples=5).fit_predict(X)
# Spectral Clustering
spectral_labels = SpectralClustering(n_clusters=n_domains, affinity="nearest_neighbors").fit(X).labels_
# Agglomerative (Single Linkage)
single_labels = AgglomerativeClustering(n_clusters=n_domains, linkage="single").fit(X).labels_

# plot results side-by-side
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
plot_clusters(axes[0], X, kmeans_labels, f"KMeans (k={n_domains})")
plot_clusters(axes[1], X, dbscan, f"DBSCAN (eps={3.0*spacing:.3f}, k={n_domains})")
plot_clusters(axes[2], X, spectral_labels, f"Spectral (k={n_domains})")
plot_clusters(axes[3], X, single_labels, f"Single Linkage (k={n_domains})")

fig.suptitle(f"Clustering Results for {f.stem} (binning='auto')", fontsize=12)
plt.tight_layout()
plt.show()
```

Discussion. k-means partitions the plane into convex (Voronoi) cells, so it cuts the ring into pieces and glues ring fragments to the nearest eye – it *cannot* represent a cluster that surrounds another one. DBSCAN and single linkage follow connectivity and find eyes, mouth and ring as separate objects whenever there is a gap between them; spectral clustering does too, because it works on the neighbourhood graph rather than on centroids. The price: DBSCAN needs a sensible `eps` and treats isolated points as noise; spectral clustering still needs `k` and is $O(n^2)$ in memory, which is why we subsampled; single linkage breaks as soon as a few stray points bridge two domains.

Task 7.2 – the noisy smileys

Repeat for the noisy images `bl_*`. Two things now matter:

1. **DBSCAN's parameters.** On the raw point cloud (`binning=1`) scan `eps` over a few multiples of the typical spacing and watch the ring, the number of clusters and the fraction of points labelled noise.
2. **Pre-processing.** The background dots are sparse and the domains are dense. Averaging blocks of pixels (`binning="auto"` , i.e. 6–10 pixels for these images) before thresholding turns "sparse dots" into light grey and "dense dots" into dark grey – it raises the signal-to-noise ratio, exactly like binning a detector image. Cluster the binned cloud with DBSCAN and with k-means and compare.

Report the number of clusters and the noise fraction for each setting. Which combination recovers the domains?

```
In [160.. # your code here
multipliers = [1.5, 3.0, 6.0]
results_raw = []

for f in noisy:
    pts_raw = image_to_points(f, binning=1)
    spacing = typical_spacing(pts_raw)

    for mult in multipliers:
        eps_val = mult * spacing
        db = DBSCAN(eps=eps_val, min_samples=5).fit(pts_raw)
        labels = db.labels_

        n_clust = n_clusters(labels)
        noise_frac = (np.mean(labels == -1) * 100) # percentage of point

        results_raw.append({"File": f.stem, "Multiplier": f"{mult}x", "ep
```



```
# display summary table for raw point cloud scan
df_raw_scan = pd.DataFrame(results_raw)
print("=== RAW CLOUD DBSCAN PARAMETER SCAN (binning=1) ===")
print(df_raw_scan.to_string(index=False))
```

```
=== RAW CLOUD DBSCAN PARAMETER SCAN (binning=1) ===
  File Multiplier      eps  Clusters Found  Noise Fraction (%)
bl_1_o      1.5x 0.0050           82      87.9
bl_1_o      3.0x 0.0100           98       8.0
bl_1_o      6.0x 0.0199            3       0.0
bl_2_o      1.5x 0.0050           76     88.6
bl_2_o      3.0x 0.0100           92       7.6
bl_2_o      6.0x 0.0199            1       0.0
bl_3_o      1.5x 0.0065          113     83.0
bl_3_o      3.0x 0.0130           36       3.0
bl_3_o      6.0x 0.0259            3       0.0
bl_4_o      1.5x 0.0068          124     81.4
bl_4_o      3.0x 0.0137           18       3.4
bl_4_o      6.0x 0.0274            1       0.0
```

Discussion. On the raw noisy clouds DBSCAN has no *clean* setting: at $1.5 \times$ spacing everything is noise, at $3 \times$ the domains shatter into hundreds of fragments, and at $6 \times$ the domains are found but nothing is rejected any more (noise 0 %) – the sparse background dots are simply glued to the nearest domain. The density contrast between domain and background is weak *at the pixel level*. After binning, the sparse background averages to light grey and vanishes at the threshold while the dense domains survive; DBSCAN then finds eyes, mouth and ring in `bl_1` and `bl_3` essentially perfectly. In `bl_2` and `bl_4` the domains *touch* (the eyes grow into the mouth, the grimace lines reach the ring), so every connectivity-based method must return one cluster – there is no gap to find. Only a method with a shape prior (k-means: convex blobs) can split them, and then it splits along the wrong lines because the shapes are not convex. The lesson generalises: **pre-processing (binning, smoothing, the choice of features) often decides more than the clustering algorithm**, and no algorithm can find a boundary that is not in the data. In a real experiment you would fix the bin size from the resolution you need (what is the smallest domain I care about?) rather than by eye.

Optional (image segmentation instead of point clouds). For a grey-scale micrograph such as `NdFeB-Domains.jpg` the observations are *pixels* and the feature is the *intensity* (plus, if you like, the position or a local texture measure such as `skimage.filters.sobel`). k-means on intensity alone is then simple thresholding into k levels – try it, and then add the pixel coordinates as extra features and observe what happens.

```
In [162... # your code here
# Load the Micrograph Image
img_path = "Smileys/NdFeB-Domains.jpg"
img = skimage.io.imread(img_path, as_gray=True)
h, w = img.shape
# Feature Set A: Intensity alone (reshaped to N x 1)
intensity_features = img.reshape(-1, 1)
# Feature Set B: Intensity + Spatial Coordinates (N x 3)
yy, xx = np.meshgrid(np.arange(h), np.arange(w), indexing="ij")
```

```

coords_intensity_features = np.column_stack([img.ravel(), xx.ravel(), yy.
# Standardize coordinates and intensity so they share comparable variance
scaler = StandardScaler()
scaled_spatial_features = scaler.fit_transform(coords_intensity_features)
# Fit KMeans (k = 3 levels/domains)
k_segments = 3
# Model 1: Intensity Only
km_intensity = KMeans(n_clusters=k_segments, n_init=10, random_state=42).
segmented_intensity = km_intensity.reshape(h, w)
# Model 2: Intensity + Spatial Coordinates
km_spatial = KMeans(n_clusters=k_segments, n_init=10, random_state=42).fi
segmented_spatial = km_spatial.reshape(h, w)
# Plot Comparison
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

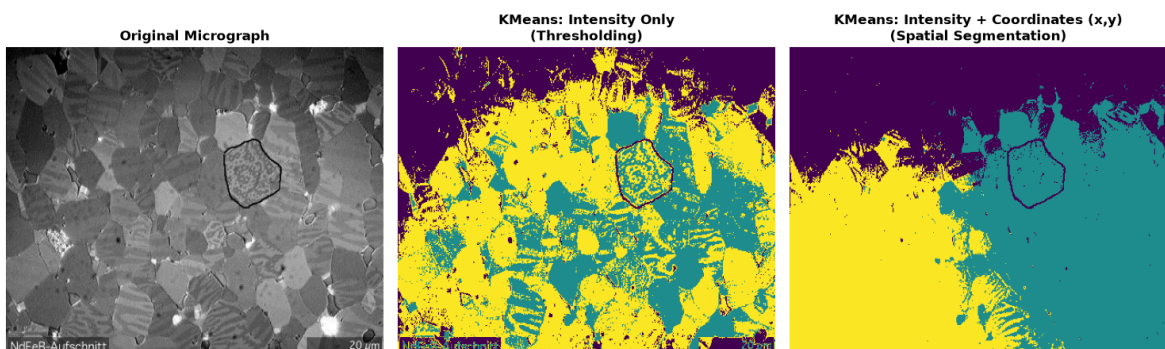
axes[0].imshow(img, cmap="gray")
axes[0].set_title("Original Micrograph", fontweight="bold")
axes[0].axis("off")

axes[1].imshow(segmented_intensity, cmap="viridis")
axes[1].set_title("KMeans: Intensity Only\n(Thresholding)", fontweight="b
axes[1].axis("off")

axes[2].imshow(segmented_spatial, cmap="viridis")
axes[2].set_title("KMeans: Intensity + Coordinates (x,y)\n(Spatial Segmen
axes[2].axis("off")

plt.tight_layout()
plt.show()

```



Part 8 (bonus) – Linear vs non-linear dimension reduction

PCA can only rotate and project. When the interesting structure lies on a curved manifold, neighbourhood-preserving methods such as **t-SNE** (and UMAP) often show it better. We compare them on the 8×8 handwritten digits (64 features, 10 classes) – a dataset where PCA to 2-D shows blobs that overlap, while t-SNE shows ten islands.

t-SNE measures pairwise similarities in high dimension with Gaussians, in low dimension with a heavy-tailed t-distribution, and moves the low-dimensional points by gradient descent until the two similarity distributions agree (Kullback–Leibler divergence). Its **perplexity** parameter sets the effective number of neighbours.

Task 8.1

Load `sklearn.datasets.load_digits()`, standardise, and plot the 2-D PCA scores next to a 2-D t-SNE embedding (`sklearn.manifold.TSNE(perplexity=30, init='pca', random_state=...)`), coloured by digit. Then vary the perplexity (5, 30, 100). Which quantities in a t-SNE map are meaningful (cluster membership) and which are **not** (cluster sizes, distances between clusters, axis orientation)?

In []: `# your code here`

Discussion. PCA keeps the global geometry (distances mean something) but the two leading components hold only about a fifth of the variance and the digits overlap. t-SNE separates the ten digits into islands at moderate perplexity, but the map is **qualitative**: the size of an island, the distance between islands and the orientation of the axes carry no information, and a different `random_state` gives a different picture. Use t-SNE/UMAP to *look* for structure, then go back to the original features (or PCA) to *quantify* it.

Summary – what you should be able to do now

1. **Explain** why PCA needs centred data and when to scale; state $PVE_j = s_j^2 / \sum s_k^2$ and read a scree plot.
2. **Derive** that SVD of a centred matrix gives the PCA loadings (V), scores ($U\Sigma$) and variances ($s_j^2 / (m - 1)$).
3. **Show** that the first PC is the rotation that minimises the perpendicular residuals, and contrast this with regression.
4. **Determine** the rank of a spectro-kinetic matrix from the scree plot, explain why the rank is $n_{\text{species}} - 1$ for difference spectra, and explain why singular vectors are not species spectra (rotational ambiguity) and how a kinetic model (global analysis) resolves it.
5. **Choose** the number of clusters with the elbow and silhouette criteria and judge a clustering against known labels with the adjusted Rand index.
6. **Read** a dendrogram and describe how the linkage choice changes the tree.
7. **Pick** a clustering algorithm from the shape of the data: centroid based (k-means) for convex blobs, density/graph based (DBSCAN, spectral, single linkage) for connected shapes, and know each one's parameters.
8. **Use** t-SNE as a qualitative tool and know which of its features are not to be interpreted.